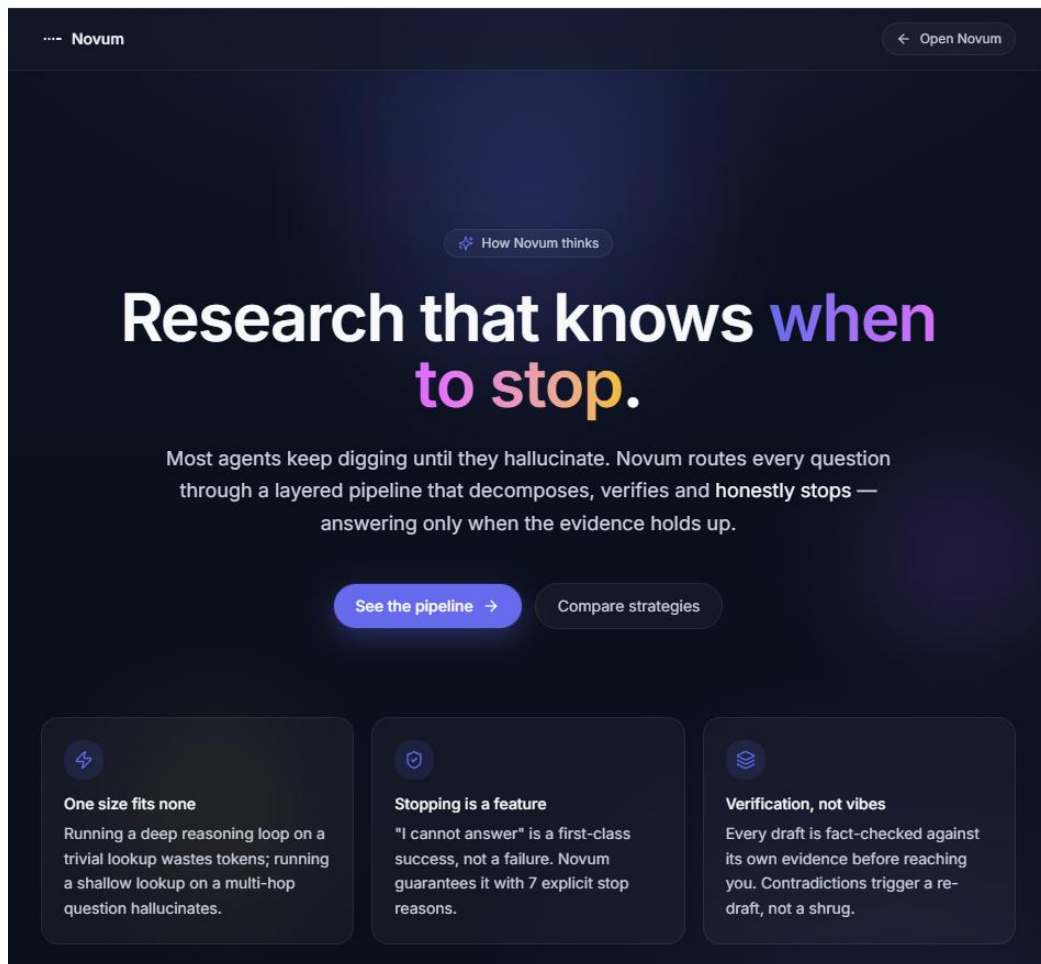


Novum

A Research Agent That Knows When to Stop

Second Deliverable: Working Build, Source Code, and Design Delta

Candidate	Giovanny Manchola
Challenge	Tekton Labs - AI Research Agent Build Challenge
Repository	https://github.com/hgmanchola/novum
Frontend	https://novum-seven.vercel.app/
Backend health	https://novum-prod.duckdns.org/health
All docs	/docs folder within the report



1. Problem Description

The challenge is titled “Build an agent that knows when to stop.” The visible requirement is simple: given a research question, the system must produce a sourced answer. The deeper challenge is more important: the system must know when it has enough evidence, make that decision defensible, and expose what happened after the run.

This is not primarily a chatbot with citations problem. A chatbot can produce a convincing answer, but that does not mean the answer was earned. A real research agent must handle ambiguity, contradictory sources, incomplete evidence, weak signals, cost limits, and the possibility that additional searching will not improve the result.

Core problem

How can an AI system produce useful research answers while making its evidence, reasoning boundaries, and stopping decision visible enough to be trusted?

Product principle

A research agent should not simply answer questions. It should earn its conclusions.

2. What I Built

Novum is a deployed full-stack AI research agent that turns open-ended questions into sourced, inspectable research runs.

- A deployed frontend where users can submit research questions and review the answer.
- A deployed backend exposing the research workflow and health endpoint.
- A source code repository with frontend, backend, architecture, research, and testing documentation.
- A trace-oriented research workflow where meaningful steps are emitted as events.
- A lane-based research strategy that adapts research depth to the type of question.
- A stopping mechanism that uses evidence, confidence, judge verification, and practical caps.
- A GUI layer that makes the run trace visible instead of hiding the process behind the final answer.
- A design layer added intentionally, because trust is communicated through experience, not only backend logic.

3. What Changed From the Initial Design Document and Why

This is the short implementation delta requested as part of the second deliverable. The changes below are not presented as mistakes. They show how the design evolved under implementation pressure.

Delta 1 - From one generic research loop to different research lanes

Initial assumption: The first design assumed that one general research loop could handle most questions, following just a single method.

What I found while building: Not every question deserves the same research depth. A direct factual question should stop quickly. A comparative question needs criteria and trade-offs. A predictive or ambiguous question needs deeper reasoning, caveats, and uncertainty handling.

Decision: I moved from a generic loop to a lane-based strategy: FAST for direct or definitional questions, STANDARD for normal comparative or technical research, and DEEP for causal, scenario-based, predictive, or highly ambiguous questions.

Consequence accepted: The orchestration became more complex, but the complexity is purposeful. It lets the system balance quality, cost, latency, and depth based on the nature of the question.

Delta 2 - From binary sufficiency to evidence-aware stopping

Initial assumption: The system could decide whether it had enough evidence through a relatively simple yes/no check.

What I found while building: Research sufficiency is rarely binary. Some answers are partially supported. Some sources conflict. Some questions are inherently uncertain. A binary mechanism either stops too early or keeps searching without a clear benefit.

Decision: I separated the stopping decision into multiple signals: evidence coverage, source agreement and contradiction, structural confidence, judge verification, value of continuing, and budget/safety caps.

Consequence accepted: The system may return a cautious, partial, or best-effort answer instead of pretending to be fully certain. For a research agent, that is the right behavior.

Delta 3 - From a final answer only to a visible GUI trace

Initial assumption: A simpler build could focus on producing the final answer and storing minimal metadata.

What I found while building: That would not satisfy the real spirit of the challenge. The evaluator needs to see what the agent did, what it found, what it concluded, and why it stopped.

Decision: I made the run trace visible in the application UI. The frontend is not just a wrapper around the backend. It is part of the trust contract.

Consequence accepted: More implementation effort went into the user experience and trace presentation. That was intentional because inspectability is not only a backend feature; it must be visible to the evaluator.

Delta 4 - From production completeness to intentional MVP boundaries

Initial assumption: It would be tempting to add production features such as full authentication, tenant management, distributed workers, advanced observability, and enterprise governance.

What I found while building: Adding all of those would dilute the core challenge. The priority was to demonstrate the reasoning engine, stopping behavior, inspectability, and extensible architecture.

Decision: I intentionally left some production capabilities out of V1 and documented them as roadmap items.

Consequence accepted: The MVP remains focused and explainable. The architecture still leaves a credible path to production, but the V1 scope prioritizes the requirements that matter most for the challenge.

4. Why the Agents Were Structured This Way

The agent architecture was not designed as a generic autonomous agent. It was structured as a set of specialized responsibilities because the challenge depends on being able to explain and audit each decision.

Classifier

Determines question type, complexity, and expected answer shape. It prevents the system from treating all questions the same.

Lane Router

Selects **FAST**, **STANDARD**, or **DEEP** so the system balances cost, latency, and research quality. Each one of them trigger different research plans.

Planner

Turns the question into sub-claims and search directions so the system knows what it is trying to support.

Plan Critic and Replanner

Prevents weak plans from wasting budget and creating false confidence.

Searcher

Gathers supporting, limiting, and contradictory evidence from external sources.

Evidence Analyzer

Evaluates coverage, agreement, contradiction, and structural confidence.

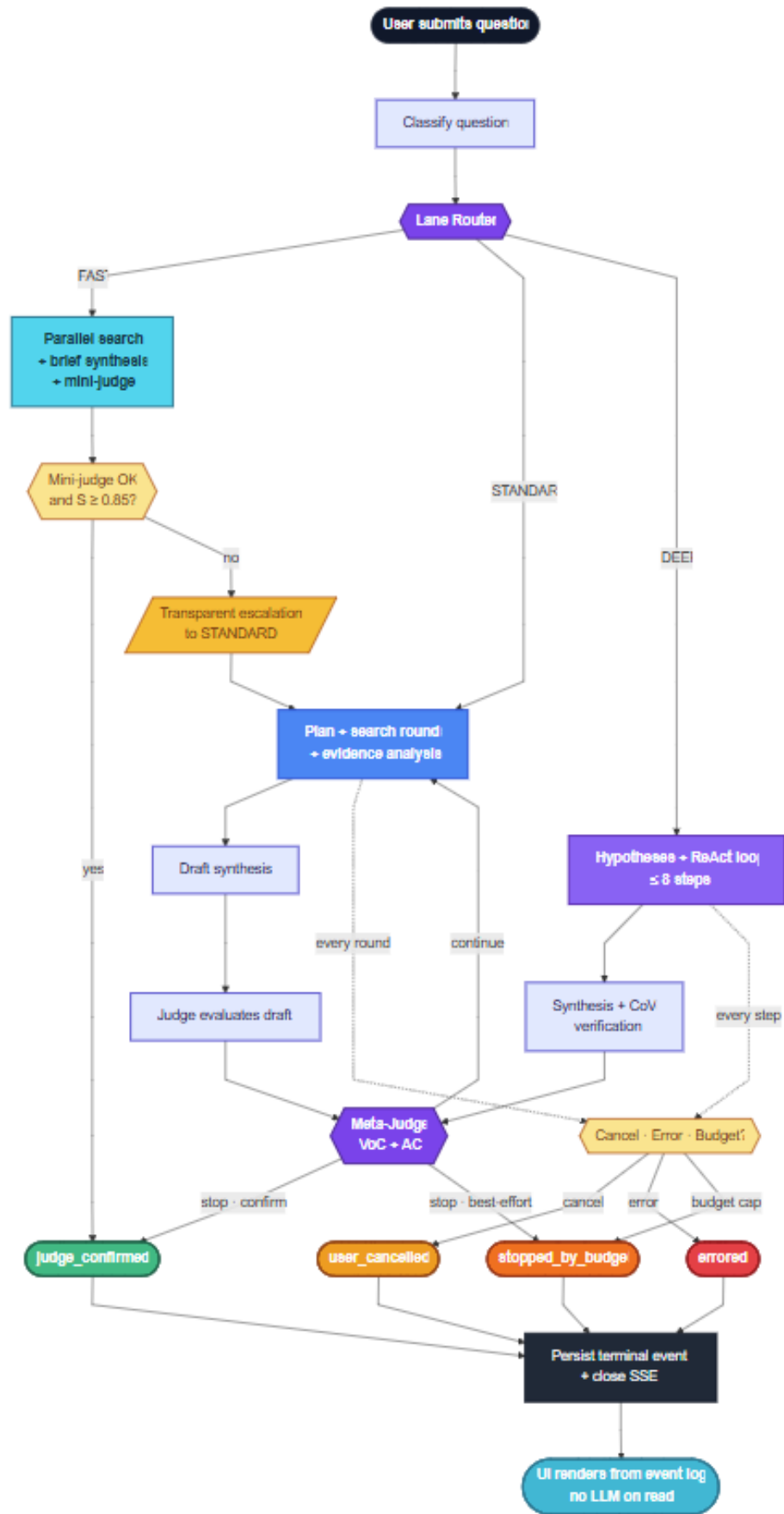
Judge and Meta-Judge

Reviews whether the answer is supported and whether more research is worth the cost.

Synthesizer and Output Renderer

Produces an answer shape appropriate for the question: direct, comparative, scenario, trade-off, or best-effort.

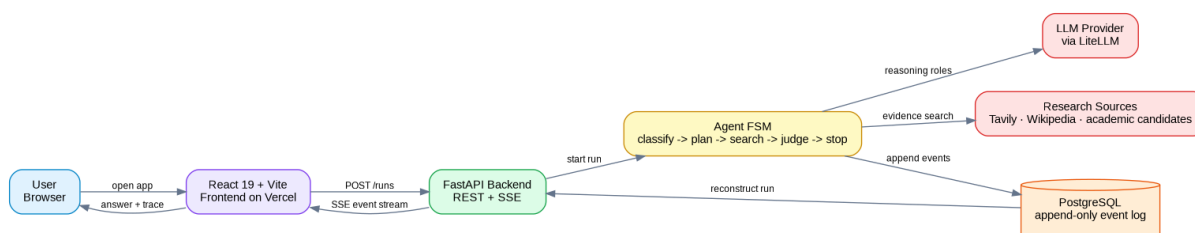
Fully lifecycle diagram located in **docs\understanding-phase\data-flows-and-diagrams.md**



5. Architecture Used

Novum uses a custom finite-state-machine research workflow, a FastAPI backend, PostgreSQL persistence, and a React frontend. The most important architectural decision is that the event log is the source of truth.

Fully Agentic architecture diagram located in **docs\understanding-phase\data-flows-and-diagrams.md**



5.1 Architectural principles

Event log is the source of truth: Every meaningful step is appended to PostgreSQL. The final answer is not isolated from the process that produced it.

Stop reasons are explicit: The stop condition is represented as a concrete terminal state, not as vague text hidden inside an LLM response.

Read determinism matters: Opening a completed run should show the same stored result. The system should not regenerate a different answer simply because the user reopened it.

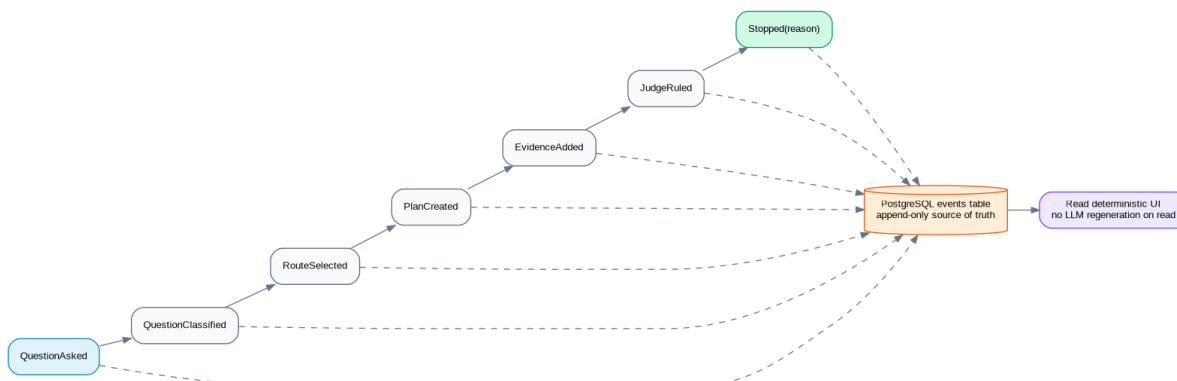
Append-only history protects trust: Events are not edited after the fact. Future resume or fork behavior should append new events instead of mutating old ones.

UI is the trust contract: The frontend exists to make the process understandable. The trace is not just a debug log; it is part of the product experience.

V1 stays intentionally bounded: The system avoids distributed complexity that is not necessary for the challenge. The design is production-aware without pretending to be a finished enterprise platform.

6. Event Log and Inspectability

The event log is what makes Novum inspectable. A completed run should show what the agent did, what it found, what it concluded, and why it stopped.



- QuestionAsked
- QuestionClassified
- RouteSelected
- PlanCreated
- PlanCritiqued
- ToolCalled
- EvidenceAdded
- ContradictionDetected
- DraftSynthesized
- JudgeRuled
- Stopped

7. How the Stop Condition Works

Problem

The agent cannot search forever, but it cannot stop only because it reached a fixed number of steps.

Tension

The tension is between completeness and cost. If the agent stops too early, the answer is shallow. If it continues too long, it wastes time, tokens, and user patience.

Decision

Novum uses evidence-aware stopping based on question type, evidence coverage, source agreement, contradictions, structural confidence, judge verification, value of continuing, and safety caps.

Consequence

The agent can stop quickly for simple questions and continue deeper for complex questions. When evidence is weak or incomplete, the answer can be shaped as best-effort instead of pretending certainty.

8. GUI and Design Layer

The frontend was not treated as a secondary concern. Although strong visual design was not the main requirement of the challenge, it was intentionally included because trust is also a product experience.

- What question was asked.
- What stage the agent is in.
- What evidence was collected.
- What the final answer is.
- Why the agent stopped.

The design layer is not intended to be a full product reveal or a complete enterprise UI. It is enough to make the trust contract visible during the review.

9. What Was Intentionally Left Out of V1

Full enterprise authentication: Deferred because the challenge evaluates research reasoning, stopping behavior, and inspectability.

Multi-tenancy and workspace governance: Deferred because V1 focuses on individual research runs.

Kubernetes or distributed orchestration: Deferred because a single-server deployment is easier to reason about and sufficient for the MVP.

Full automated evaluation pipeline: Deferred because the prepared test questions can become golden traces and CI quality gates later.

Full fork/replay UX: Prepared through append-only events and meaningful fork points, but a complete branching interface is V2.

Cross-provider judge independence: Prepared through provider abstraction, but V1 focuses on proving the workflow.

10. Technical Stack Summary

Frontend

React 19, Vite, TypeScript, Tailwind, shadcn/ui, Zustand, TanStack Query, and native EventSource. The frontend needs to be responsive, fast, and clear.

Backend

FastAPI, Python 3.12, Pydantic, SQLAlchemy, Alembic, and PostgreSQL. Python fits the AI ecosystem and PostgreSQL provides durable event storage.

AI orchestration

Custom finite-state machine with structured LLM roles and provider abstraction through LiteLLM. The custom FSM was chosen because every transition can be defended directly.

Deployment

Vercel for the frontend and a backend server with FastAPI, Caddy, PostgreSQL, and systemd-managed services. This is appropriate for the challenge MVP.

Appendix A - Submission Checklist

Frontend

- Landing page explains what Novum does.
- Reviewer can run a real question quickly.
- Trace panel is obvious.
- Final answer is visually separated from trace.
- Errors are readable.

Backend

- Health endpoint is reachable.
- Run creation works.

- Event streaming works.
- Terminal events are persisted.
- Logs are available for debugging.

Repository

- README explains how to run the application.
- README includes What Changed From the Design Doc and Why.
- Architecture, tech stack, research model, and test questions are easy to find.
- Known limitations are stated honestly.

Demo

- Prepare one trivial question.
- Prepare one ambiguous question.
- Prepare one technical trade-off question.
- Prepare one deep research question.

Appendix B - Key Links

Repository: <https://github.com/hgmancholan/novum>

Frontend: <https://novum-seven.vercel.app/>

Backend health: <https://novum-prod.duckdns.org/health>

Challenge brief: <https://github.com/hgmancholan/novum/blob/main/docs/understanding-phase/agent-challenge-candidate.html>

Architecture: <https://github.com/hgmancholan/novum/blob/main/docs/technical-phase/architecture.md>

Tech stack: <https://github.com/hgmancholan/novum/blob/main/docs/technical-phase/tech-stack.md>

Research model: <https://github.com/hgmancholan/novum/blob/main/docs/understanding-phase/advanced-ai-research.md>

Data flows and diagrams:

<https://github.com/hgmancholan/novum/blob/main/docs/understanding-phase/data-flows-and-diagrams.md>

Test questions: <https://github.com/hgmancholan/novum/blob/main/docs/q-for-testing.md>